

# Offline-first sync

An outbox, a drain loop, and one conflict rule

**Dinu-Ștefan Rusu**

FILS, Universitatea Politehnica București · Week 10

# What this lab is for

---



## Write once, sync later

Every note write commits to the notes table and an outbox row, in one transaction. No edit is ever lost to a dropped connection.

### TIME

2 hours



## Resolve one conflict, visibly

Two devices editing the same note converge on the server's version, and the device that lost the race shows a marker.

### SUBMIT

Push to your team repo, branch `lab-5`, before next lab

# Where your code goes

---

1. Branch the repository, add Drift, and generate the boilerplate:

```
git checkout -b lab-5
flutter pub add drift sqlite3_flutter_libs path_provider http
flutter pub add -d drift_dev build_runner
dart run build_runner build
```

2. Local code lives in `lib/lab5/`. Server code lives in `server/lab5/main.go`.

## Regenerate after every schema change

Adding a column to Notes or Outbox needs a fresh build and a bumped schema version, or an existing install crashes on the next launch.

# Schema and outbox, one transaction

---

1. Model `Notes`: `id`, `title`, `body`, `updatedAt`, `version`, `isDirty`, `elsewhere`, `deletedAt`.
2. Model `Outbox`: `id`, `entityId`, `op`, `payload`, `queuedAt`.
3. Write `saveNote` so both tables commit inside one `db.transaction`.
4. Make delete a write: set `deletedAt`, never issue a SQL DELETE.

## DONE WHEN

- ☐ Editing a note updates `notes` and inserts one `outbox` row.
- ☐ Killing the app mid-save leaves either both rows, or neither.
- ☐ Deleting a note still leaves an `outbox` row for the delete.

---

**A write is not durable until the outbox has it too.** An `outbox` row inserted outside the transaction can vanish if the app dies between the two writes.

# Two tables, one purpose

---

```
class Notes extends Table {  
    late final id = text();  
    late final title = text();  
    late final body = text();  
    late final updatedAt =  
        dateTime();  
    late final version = integer();  
    late final isDirty = boolean();  
    late final elsewhere = boolean();  
    late final deletedAt =  
        dateTime().nullable();  
}
```

```
class Outbox extends Table {  
    late final id = text();  
    late final entityId = text();  
    late final op = text();  
    late final payload = text();  
    late final queuedAt = dateTime();  
}
```

## Both writes, one transaction

---

```
Future<void> saveNote(Note n) => db.transaction(() async {  
    final row = n.copyWith(  
        updatedAt: DateTime.now(), isDirty: true);  
    await into(notes).insertOnConflictUpdate(row);  
    await into(outbox).insert(OutboxCompanion.insert(  
        id: uuid.v4(), entityId: n.id, queuedAt: DateTime.now(),  
        op: 'upsert', payload: jsonEncode(row.toJson()),  
    ));  
});
```

watch() on notes fires the moment this commits. The outbox row leaves later, but the edit is already on screen.

# Drain the outbox, then pull the delta

---

1. Add `drainOutbox()`: read the outbox oldest first, POST each payload.
2. On success, delete the row and clear `isDirty` in one transaction.
3. On start, and when connectivity returns, call `pull()` with the last saved version.
4. Trigger both from a connectivity listener, never a timer.

## DONE WHEN

- ☐ Airplane mode on, edit, airplane mode off: the row leaves the outbox within a few seconds.
- ☐ Restarting with a non-empty outbox drains it before the list refreshes.
- ☐ `pull()` never re-downloads a row it already has.

# The drain loop

---

```
Future<void> drainOutbox() async {  
    final batch = await (select(outbox)  
        ..orderBy([(o) => OrderingTerm.asc(o.queuedAt)]))  
        .get();  
    for (final item in batch) {  
        final r = await http.post(Uri.parse('$base/notes'), body: item.payload);  
        if (r.statusCode != 200) break;    // stop, keep order  
        await db.transaction(() async {  
            await (delete(outbox)..where((o) => o.id.equals(item.id))).go();  
            await notesDao.clearDirty(item.entityId);  
        });  
    }  
}
```



# Delta sync on start

---

```
Future<void> pull() async {  
    final since = await meta.lastVersion();  
    final uri = Uri.parse('$base/notes?since=$since');  
    final rows = jsonDecode((await http.get(uri)).body) as List;  
    var newest = since;  
    await db.transaction(() async {  
        for (final row in rows) {  
            await notesDao.upsertFromServer(row);  
            newest = max(newest, row['version'] as int);  
        }  
    });  
    await meta.setLastVersion(newest);  
}
```

# The whole sync server, part 1

---

```
type Note struct{ ID, Title, Body, UpdatedAt, DeletedAt string; Version int }
var mu sync.Mutex; var version int
var notes = map[string]Note{}
func upsert(w http.ResponseWriter, r *http.Request) {
    var n Note
    json.NewDecoder(r.Body).Decode(&n)
    mu.Lock()
    version++
    n.Version = version
    notes[n.ID] = n
    mu.Unlock()
    json.NewEncoder(w).Encode(n)
}
```

## The whole sync server, part 2

---

```
func list(w http.ResponseWriter, r *http.Request) {
    since, _ := strconv.Atoi(r.URL.Query().Get("since"))
    mu.Lock()
    defer mu.Unlock()
    var out []Note
    for _, n := range notes {
        if n.Version > since {
            out = append(out, n)
        }
    }
    json.NewEncoder(w).Encode(out)
}
```

main() wires POST and GET on /notes to these handlers, then listens on :8080.

# One conflict rule, and a visible marker

---

1. On pull, if the local row is not dirty, upsert the server's row: no conflict.
2. If the local row is dirty and the incoming version is higher, the server wins: keep its value.
3. Set `elsewhere` on the losing row instead of silently overwriting it.
4. Clear the marker once the user opens the note.

## DONE WHEN

- ☐ Two devices editing the same note end with one value on both.
- ☐ The device that lost the race shows an "Updated elsewhere" badge.
- ☐ The badge clears once the note is opened.

---

**The loser is not the older edit.** It is the device that had not yet seen the version that won.

# The rule: server version wins

---

```
Future<void> onIncoming(Map row) => db.transaction(() async {  
    final local = await notesDao.find(row['id'] as String);  
    if (local == null || !local.isDirty) {  
        return notesDao.upsertFromServer(row);    // no conflict  
    }  
    if ((row['version'] as int) > local.version) {  
        await notesDao.markUpdatedElsewhere(local.id);  
        return notesDao.upsertFromServer(row);    // we lost  
    }  
    // local.version >= incoming: we are still ahead, keep ours  
});
```

Both devices run this same function, so both reach the same answer without talking to each other.

## The marker in the list

---

```
ListTile(  
  title: Text(note.title),  
  trailing: note.elsewhere  
    ? const Chip(label: Text('Updated elsewhere'))  
    : null,  
  onTap: () => notesDao.clearFlag(note.id),  
)
```

---

**The marker is the whole UI cost of this rule.** No merge screen, no dialog: one boolean and one badge tell the user their edit did not survive.

# What the grader will do

---

## THE DEMO

- ☐ Airplane mode on, edit a note, airplane mode off: the server has the edit within a few seconds.
- ☐ Two devices, or two emulators, edit the same note. Both end up with the same value.
- ☐ The device that lost the race shows the "Updated elsewhere" marker.

## IN THE REPOSITORY

The Drift schema, the sync engine, and `server/lab5/main.go`, with a README naming your conflict rule.

One commit per task. A screenshot of both devices showing the same value is the cheapest proof you can leave.

# Four failures you will meet

---

Target of URI hasn't been generated: app\_db.g.dart

build\_runner has not run, or ran before the table file was saved. Re-run it, then restart analysis.

---

Connection refused, or the request hangs until it times out

The emulator's `localhost` is itself. Point the client at `10.0.2.2`, not `localhost`.

---

The outbox row count never drops to zero

An exception is swallowed silently, or the loop never runs because the connectivity check never fires. Log every iteration.

---

A device keeps re-downloading rows it already has

The checkpoint was saved before the transaction committed, or the comparison used `>=` instead of `>`.

---



# Push before you leave.

Branch lab-5 · one commit per task · your conflict rule named in the README